

VITZON

VIT Marketplace — Software Design Document (SDD)

Version: 1.0 **Status:** MVP Build Specification **Date:** 18 September 2026

A campus-only marketplace and lost-and-found platform for VIT students. This document is an implementation-ready specification for a coding agent such as Codex.

1. Executive Summary

VITZON is a student-to-student campus marketplace for buying and selling second-hand items such as mattresses, textbooks, lab coats, calculators, electronics, stationery and hostel essentials. It also includes a Lost & Found module for reporting and discovering misplaced items.

The MVP should prioritize trust, simplicity, fast discovery and safe campus-local interactions. Firebase is the backend platform for authentication, database, storage and server-side enforcement; Tailwind CSS is used for UI styling.

2. Product Vision and Objectives

Vision: Make it easy for VIT students to reuse, sell, buy and recover items within their campus community.

- Allow verified VIT students to authenticate with their institutional email.
- Let students create, edit, pause and delete marketplace listings with a price and item details.
- Enable buyers to search, filter, sort and view listings.
- Provide a campus-local contact flow without exposing unnecessary personal information.
- Allow users to report lost and found items and search active reports.
- Provide moderation/reporting controls to reduce scams, prohibited listings and abuse.

3. Scope

3.1 MVP — In Scope

- VIT email-based authentication.
- Student profile with display name and basic contact preference.
- Marketplace home/feed.
- Categories, search, filters and sorting.
- Create/view/edit/delete listing.
- Multiple item photos using Firebase Storage.
- Price, condition, description, category and pickup/location information.
- Listing status: ACTIVE, RESERVED, SOLD, EXPIRED.
- Buyer-to-seller contact request using an in-app request flow.
- Favorites/saved listings.
- Lost & Found create/view/search/report flows.
- Report listing/user/report reason.
- Basic admin moderation.
- Responsive mobile-first UI.

3.2 Explicitly Out of Scope for MVP

- Online payments, escrow or checkout.
- Shipping or delivery.
- Public access outside the VIT community.
- Complex real-time chat.

- AI-based price prediction or image recognition.
- Multi-campus federation.
- Social-feed features unrelated to buying/selling or Lost & Found.

4. Assumptions and Product Decisions

Area	Decision
Identity	Only approved VIT institutional email domains may register/login.
Password	Firebase Authentication manages credentials; the application never stores raw passwords.
Backend	Firebase Authentication + Firestore + Storage; Cloud Functions only where privileged/server-side logic is required.
Images	Stored in Firebase Storage; Firestore stores metadata and paths.
Transactions	Offline campus handover; VITZON does not process payment in MVP.
Trust	Authenticated users, reporting, moderation, listing status and minimal profile exposure.
Campus location	Use selectable campus/hostel/pickup zones rather than exact residential addresses.

5. User Roles

Student

- Register/login using VIT email.
- Manage profile.
- Browse/search/favorite listings.
- Create and manage own listings.
- Request seller contact.
- Create and search Lost & Found reports.
- Report suspicious listings/users.

Moderator/Admin

- View reported listings/users/reports.
- Hide or restore marketplace listings.
- Review Lost & Found reports.
- Suspend users when required.
- View audit records for moderation actions.

6. Core User Stories

- As a VIT student, I want to sign in using my institutional account so that the marketplace remains campus-only.
- As a seller, I want to create a listing with photos, title, description, condition and price so other students can discover my item.
- As a buyer, I want to search and filter listings so I can quickly find a relevant item.
- As a buyer, I want to see whether an item is active, reserved or sold so I know whether it is still available.

- As a seller, I want to mark an item sold/reserved so the listing state remains accurate.
- As a buyer, I want to save listings so I can revisit them later.
- As a student, I want to report a listing so inappropriate or fraudulent content can be reviewed.
- As a student who lost something, I want to post a Lost report with enough identifying information to help recovery.
- As a student who found something, I want to post a Found report so the owner can identify and claim it.

7. Functional Requirements

FR-01 Authentication

- Support email/password authentication through Firebase Authentication.
- Registration must validate the email against an allowlisted VIT domain.
- Email verification is required before marketplace actions are enabled.
- Login, logout and password reset must be supported.
- Authenticated identity must be derived from Firebase Auth UID, never from client-supplied UID fields.
- If multiple VIT domains are required, configure an allowlist rather than hard-coding assumptions in UI code.

FR-02 Profile

- Required: displayName, institutional email, createdAt.
- Optional: profile photo, phone/contact preference.
- Do not expose private email/phone publicly unless the user explicitly chooses a contact method.
- Account deactivation must preserve required audit references without exposing unnecessary PII.

FR-03 Marketplace

- Listing fields: title, category, description, price, condition, campus zone, pickup preference, image references, seller UID, status, createdAt, updatedAt.
- Initial categories: Books, Hostel, Clothing, Electronics, Lab, Stationery, Sports, Other.
- Price must be non-negative and denominated in INR.
- Only ACTIVE listings appear in default discovery.
- Listing owner can edit/delete/change status.
- SOLD/EXPIRED listings are retained for audit/history but excluded from normal discovery.
- Security rules must prevent users from editing another user's listing.

FR-04 Search, Filter and Sort

- Search by title/category using a search index if scale requires it; do not implement expensive full-collection client filtering.
- Filters: category, price range, condition, campus zone, status.
- Sort: newest, price low-to-high, price high-to-low.
- Use pagination/infinite scrolling.

FR-05 Contact Request

- Buyer sends a short contact request message.
- Seller accepts/declines.
- Only participants can read request details.
- Do not expose exact personal location by default.
- MVP uses a request/inbox model instead of full real-time chat.

FR-06 Favorites

- Users can save/unsave active listings.
- Favorites are private.
- Deleted/inaccessible listings should appear as unavailable and be removable.

FR-07 Lost & Found

- Report type: LOST or FOUND.
- Fields: title, category, description, approximate location/zone, date/time range, image, identifying details, reporter UID, status, createdAt, updatedAt.
- Statuses: OPEN, MATCHED, RESOLVED, CLOSED.
- Search/filter by report type, category, location zone, date and status.
- Avoid publishing highly sensitive identifying details that make false claims easier.
- Use a verification-oriented claim/contact flow before returning an item.

FR-08 Reporting and Moderation

- Users can report listings/users/Lost & Found reports.
- Reasons: scam/fraud, prohibited item, harassment, misleading information, duplicate, inappropriate content, other.
- Reports enter a moderation queue.
- Moderators can mark OPEN, REVIEWING, RESOLVED or DISMISSED.
- Moderation actions create an audit record.

8. Non-Functional Requirements

- Performance: fast initial rendering on typical student mobile networks; paginate large collections.
- Security: enforce authorization in Firebase Security Rules, not only frontend logic.
- Availability: use Firebase managed services and avoid unnecessary custom backend dependencies for MVP.
- Responsive design: mobile-first; support desktop browsers.
- Accessibility: keyboard navigation, visible focus, semantic controls, adequate contrast, labels and meaningful alt text.
- Reliability: explicit error handling for writes and safe retries for transient reads.
- Observability: capture client errors and key backend failures; maintain moderation audit logs.

- Data minimization: collect only information needed for campus marketplace operation.
- Maintainability: feature-based frontend structure, typed models, centralized validation and reusable UI components.

9. Proposed Architecture

Frontend: React + Tailwind CSS (recommended implementation choice).

Firebase: Firebase Authentication, Cloud Firestore, Firebase Storage, optional Cloud Functions, Firebase App Check where appropriate.

Flow: Browser → React UI → Auth context → feature services/hooks → Firebase SDK → Auth/Firestore/Storage. Privileged operations → Cloud Functions/Admin SDK.

The client must never contain Firebase Admin credentials or service-account keys.

10. Suggested Repository Structure

src/app/routes, src/app/providers, src/components/ui, src/components/forms, src/components/layout, src/features/auth, src/features/marketplace, src/features/favorites, src/features/contactRequests, src/features/lostFound, src/features/moderation, src/features/profile, src/lib/firebase, src/lib/validation, src/lib/utils, src/types, src/styles, src/tests, functions, firestore.rules, firestore.indexes.json, storage.rules

Keep business rules in feature services/validators rather than embedding complex logic in presentation components.

11. Firestore Data Model

users/{uid}: displayName, email, photoURL, role, contactPreference, campus, createdAt, updatedAt, status

listings/{listingId}: sellerId, title, normalizedTitle, categoryId, description, priceInr, condition, campusZoneId, pickupPreference, imageRefs[], status, createdAt, updatedAt

favorites/{uid}/items/{listingId}: createdAt

contactRequests/{requestId}: listingId, buyerId, sellerId, message, status, createdAt, updatedAt

lostFoundReports/{reportId}: reportType, reporterId, title, categoryId, description, campusZoneId, dateObserved, imageRefs[], status, createdAt, updatedAt

reports/{reportId}: targetType, targetId, reporterId, reason, details, status, moderatorId, resolution, createdAt, updatedAt

moderationAudit/{auditId}: actorId, action, targetType, targetId, reason, metadata, createdAt

categories/{categoryId}: name, type, active, sortOrder

12. Firestore Security Rules — Required Principles

- Unauthenticated users cannot read/write private application data.
- Only verified VIT users may create marketplace/Lost & Found content.
- A listing can be created only with sellerId == request.auth.uid.
- Only owner or authorized moderator can update/delete a listing.
- Users can only create/read their own favorites.
- Contact requests can only be read by buyer, seller or authorized moderator.
- Normal users cannot set role=admin/moderator, moderation status, audit fields or another user's UID.

- Prefer server timestamps for createdAt/updatedAt.
- Validate field types, lengths, enums and numeric ranges in rules as well as frontend validation.
- Storage rules must restrict uploads to authenticated users and validate path ownership, content type and size limits.

13. UI / UX Specification

Primary Navigation

Home • Marketplace • Sell • Lost & Found • Favorites • Requests • Profile

Marketplace Screens

- Marketplace feed: search bar, category chips, filters, sort, listing cards.
- Listing details: gallery, title, price, condition, seller summary, pickup zone, status, favorite and contact actions.
- Create/edit listing: section-based form with image upload, validation and preview.
- My listings: Active, Reserved, Sold and archived states.

Lost & Found Screens

- Lost & Found feed with LOST/FOUND toggle.
- Report detail page.
- Create report form.
- My reports page.
- Resolution/claim flow.

Required States

Loading skeleton • Empty state • Error state with retry • Success feedback • Permission denied • Deleted/unavailable.

14. Validation Rules

- Title: 5–100 characters.
- Description: 10–2,000 characters.
- Price: INR value ≥ 0 and $\leq$ configured maximum.
- Category: must exist and be active.
- Condition: NEW, LIKE_NEW, GOOD, FAIR, POOR.
- Campus zone: must be from configured allowed values.
- Images: common image formats only with a reasonable size limit and safe storage paths.
- Report details must be length-limited and sanitized.
- Render user content as escaped text; never interpret it as HTML.

15. Search and Indexing

Firestore supports structured filters and sorting but is not a general-purpose full-text search engine. For MVP, use Firestore-compatible prefix/keyword strategies only if the dataset remains small and the UX is acceptable. If full-text

search is required, introduce a dedicated search index behind an adapter. Do not download all listings to the browser and filter them there.

Create composite Firestore indexes only for query combinations actually used by the UI. Keep the index inventory in `firestore.indexes.json`.

16. Image Storage Design

Recommended paths: `marketplace/listings/{listingId}/{imageId}` and `lost-found/reports/{reportId}/{imageId}`. Enforce authenticated ownership during upload. Store references/metadata in Firestore. Generate appropriately sized previews and avoid unnecessarily large uploads.

17. Error Handling

- Centralize Firebase error mapping into user-safe messages.
- Never expose raw Firebase/internal errors.
- Retry transient reads where appropriate.
- Preserve form state after failed writes.
- For authorization failures, show a clear permission message.
- For missing/deleted records, render an unavailable state.

18. Security and Abuse Controls

- Institutional email verification.
- Firebase Security Rules as authoritative authorization.
- Evaluate Firebase App Check for production.
- Protect expensive/server-triggered actions with Cloud Functions/rate controls where needed.
- Moderate prohibited content and suspicious behavior.
- Do not expose exact hostel room numbers or precise residential addresses by default.
- Do not store passwords in Firestore.
- Do not commit secrets to source control.
- Use least-privilege Firebase roles.

19. Analytics / Product Metrics

Collect privacy-conscious aggregate metrics where permitted:

- Verified registrations and activation rate.
- Active listings created per week.
- Listing view-to-contact-request rate.
- Time from listing creation to SOLD/RESOLVED.
- Search/filter usage.
- Lost vs Found reports and resolution rate.
- Reports requiring moderation and moderation time.

Document retention and consent requirements before production launch.

20. Acceptance Criteria

ID	Acceptance criterion
AC-01	Verified VIT student can register/login and reach marketplace.
AC-02	Non-allowlisted email cannot create an account.
AC-03	Seller can create a valid listing with core required fields.
AC-04	One user cannot edit/delete another user's listing.
AC-05	Discovery supports search/filter/sort without loading the full collection.
AC-06	Buyer can favorite and unfavorite a listing.
AC-07	Buyer can submit a contact request and seller can accept/decline.
AC-08	Seller can mark listing RESERVED or SOLD and discovery respects status.
AC-09	Student can create and search LOST/FOUND reports.
AC-10	User report enters moderator queue.
AC-11	Moderator action creates an audit record.
AC-12	Unauthorized Firestore/Storage writes are denied even if frontend is bypassed.
AC-13	Primary screens have loading, empty, error and unavailable states.
AC-14	Application is usable on mobile and desktop widths.

21. Testing Strategy

- Unit tests: validators, formatters, permission helpers and pure business logic.
- Component tests: forms, listing cards, filters and state variants.
- Integration tests: Firebase Emulator for Auth, Firestore and Storage rules.
- End-to-end tests: auth, create listing, search, favorite, contact request, Lost & Found, moderation.
- Security tests: unauthorized reads/writes against Firebase Emulator Suite.
- Regression suite on every pull request.

22. Delivery Plan

Phase	Deliverables
1. Foundation	React app, Tailwind, Firebase config, routing, shared UI, lint/test setup.
2. Auth/Profile	VIT email allowlist, Auth flows, verification, profile and protected routes.
3. Marketplace	Listing CRUD, image uploads, feed, details, filters, status management.
4. Engagement	Favorites and contact-request workflow.
5. Lost & Found	Report CRUD, search/filter, resolution flow.
6. Moderation	Reports queue, moderation actions, audit logs.
7. Hardening	Rules tests, performance, accessibility, error handling, App Check evaluation, deployment.

23. Codex Implementation Instructions

- Treat this SDD as the source of truth for MVP scope.
- Implement in small, reviewable increments.
- Use TypeScript if permitted; define shared domain types.
- Use Firebase Emulator Suite during local development and security-rule testing.
- Never bypass Firestore Security Rules with client-side assumptions.
- Centralize Firebase initialization and typed service functions.
- Create reusable form controls and validation schemas.
- Prefer server timestamps and deterministic IDs where appropriate.
- Add tests alongside each feature.
- Do not implement payment, shipping or external public marketplace access unless explicitly added later.

24. Definition of Done

- Feature works against Firebase Emulator Suite.
- Security rules cover every new collection/storage path.
- Happy path and key failure paths have automated tests.
- Responsive UI implemented.
- Accessibility checks completed.
- No secrets committed.
- Firestore indexes committed.
- User-facing errors are understandable.
- Moderation/audit requirements implemented for privileged actions.
- Production build succeeds with no blocking lint/type/test errors.

25. Recommended MVP Release Checklist

- Create separate Firebase development and production projects.
- Configure institutional email allowlist.
- Enable email verification and password reset.
- Deploy and test Firestore/Storage rules.
- Review public-facing fields for privacy minimization.
- Seed categories and campus zones.
- Create initial moderator accounts through controlled access.
- Test abuse/report workflows.
- Run mobile and desktop QA.
- Run Firebase Emulator security tests.
- Enable production monitoring/error reporting.
- Publish Terms/Community Guidelines and clarify that VITZON does not process payments in MVP.

26. Future Roadmap — Post-MVP

- Real-time chat with abuse controls.
- Push notifications/email notifications.
- Verified student profile badges.
- Campus-specific communities/hostel groups.
- Advanced full-text search.
- Price history and suggested pricing.
- Duplicate Lost & Found matching assistance.
- Admin analytics dashboard.
- Optional payment/escrow only after legal, financial and institutional requirements are defined.

27. Final Product Definition

VITZON MVP is complete when a verified VIT student can securely enter the application, discover campus listings, buy/sell through a student contact workflow, manage their own listings, and use Lost & Found to report and discover misplaced items—while Firebase Security Rules, moderation and audit controls protect the platform.

Implementation priority: Security → correctness → core marketplace flow → Lost & Found → usability → optimization → future features.